

BoilerBridge: Sprint 2 Retrospective

Team 26 - BoilerBridge: Xavion Marable, Logan Wilkinson, Emilio Guzman,
Shashank Mukkera, Shreyansh Tehanguria

1. What went well? (0.5 point)

- a. You may write this section in sentences and/or list successful user stories and tasks with detailed discussions.
- The backend API for the shared costs and cost splits turned out to be well designed. After fixing a couple of routing issues, all of the CRUD operations were implemented for `/api/groups/:groupId/shared-costs` and `/api/groups/:groupId/cost-splits`. All relevant fields were enforced to be present, all amounts were enforced to be positive, and group participants were correctly validated. For create, the creator of the group or the group admins could make changes. For edit/deletes, only the creator of the group or the group admins could make changes. The ledger functionality was updated to correctly update the group balances when a new shared cost was created. **(User Stories #9 and #10)**
- The expansion of the "live trip" core was a big win this sprint. We successfully implemented the SOS Button (User Story #1) with a localized database and offline caching, so emergency numbers stay accessible even without a data connection. I also built out the Group Board (User Story #2) infrastructure, including the pinned announcements array and real-time update logic. The Role Assignment (User Story #3) system is fully integrated via a custom junction model, which handles leader, admin, and viewer permissions. I also finished the Travel Reminders (User Story #7) and Rainy Day

Plans (User Story #8) logic on the backend, which are fully functional and ready for UI re-integration.

- I really like how the Photo Album UI and functionality turned out overall (User Story #11). The implementations of the backend functions originally were slow and stored large base64 strings for images in our main database. I was able to confer with another group member about what they used for photo storage as he had already implemented user profile pictures. Through this I was able to change implementations of these backend functions to be more efficient using cloudinary image storage. The ability to delete and load all of these images upon visiting the tab in any groups page worked without a hitch and allowed easy implementation of frontend backend api calls. The implementation of creating and voting on polls (User Story #17) in groups also turned out really well. The UI turned out really well and integrated well with the way the data from the backend was formatted. I was able to successfully implement only allowing a single vote per poll per person as well as a count next to each option indicating how many votes have been cast on those options.

2. What did not go well? (0.5 point)

- a. Include general retrospective review for this sprint.
 - b. Ensure to list ALL unsuccessful user stories and tasks with detailed discussions. (They should be in line with the Sprint Planning Document for the next sprint.)
- Fixed issue where shared costs and cost splits endpoints were not routing correctly which meant the application would fail. The group ID in the URL path is a custom UUID field (groupId) on the TravelGroup document, not the MongoDB _id. Initially all routes had been set up using `TravelGroup.findById(groupId)` which caused a `CastError` for every request until the root cause had been found and all routes updated to use `TravelGroup.findOne({ groupId: groupId })`. **(User stories #9 and #10)**

- When I first set up the test suite it was set up to test against the production database instead of the test database. In a couple of the test files I had not set process.env.MONGODB_URI and therefore it was not being defaulted to process.env.TEST_MONGODB_URI which meant that User.deleteMany({}) and TravelGroup.deleteMany({}) were running against the real database during test runs because dbConnect remembers the URI that it connects with when the module loads it.

(User Stories #9 and #10)

- While working on User Story #4 (External Booking) and User Story #5 (Vibe Tags), I hit some major environment config issues. My local setup wasn't isolating API keys correctly, which caused a lot of 401 errors during integration testing. I also had to spend like 5 hours nuking over 200 global linting errors that were blocking husky hooks for the whole team because the eslint 9 flat config was going insane with our legacy rules. Most annoying was that some of my UI work for the external booking and rainy day checklist disappeared after a messy merge, so those features are currently backend-only until we can re-add the buttons and frontend logic.
- While working on what was originally meant to be SMS notifications(User Story #13) to group members from the group leader I ran into a big problem. There is no way to set up and demo SMS capabilities in our web application. To make use of automated SMS messages we would need to register a company and open an actual account under that business due to regulations around automated SMS messaging. I then had to pivot to making this functionality but for email notifications instead due to this roadblock. Luckily we had just gotten automated email sending working and it wasn't hard to pivot but inevitably lost us points in our demo due to not matching requirements.
-

3. How should you improve? (1.0 point)

- a. Mention at least two ways to improve your work in the next sprint and be as detailed as you can.
- We should establish a routing convention at the start of the next sprint. As a matter of principle, since the app uses custom UUID fields for groups, trips, etc and these are never equal to the MongoDB `_id`, all developers should know to use `findOne({ fieldName: param })` instead of `findById`. We should write a shared utility function to look up a trip and then reuse that function in the endpoints that hit this bug.
- I think the pre-run safety check in `jest.setup.js` should also assert that `MONGODB_URI` is set to point to the test database. This would mean that no test file can accidentally wipe the production database even if they are trying to use the override incorrectly (and it would error before that incorrectly overridden code could even run)
- We need to automate the environment validation process. We're going to write a script that runs before `npm start` to verify that all the API keys like weather, bag storage, and maps are actually present and valid in the `.env` so we don't deal with silent failures anymore.
- We're going to tighten up the code review workflow by enforcing a "clean-as-you-go" policy for linting. Instead of waiting until the end of the sprint to fix types and formatting, we'll set up a github action to run the linter on every PR. If the warning count goes up, it blocks the merge so the technical debt doesn't snowball and mess up our deployment pipeline again.

4. Notes

- a. For section 1 and 2, only listing out some user stories and tasks without any discussion will not get you full credits.
- b. Please make sure to include your project name, team number and the names of all your team members.

Total: 2 points (2% of your project grade)